

Git och GitHub från grunden

Tutorial – JavaScript, VS Code och GitHub

Total tid: 180 minuter

Nivå: Nybörjare

Verktyg: VS Code, Git och GitHub

Programmeringsspråk: JavaScript

Lärandemål:

Efter denna genomgång ska du kunna:

- förklara varför versionshantering används
- förstå skillnaden mellan Git, GitHub och VS Code
- skapa ett GitHub repository
- klona ett repository till din dator
- göra ändringar i ett JavaScript-projekt
- skapa en commit
- pusha ändringar till GitHub
- se projektets historik

Del 1 Varför versionshantering?

25 minuter

Problemet

Tänk dig att du arbetar med ett JavaScript-projekt.

Du har en fil:

script.js

Du arbetar med projektet under flera dagar.

Först fungerar koden:

```
const name = "Anna";
```

```
console.log(`Hello ${name}!`);
```

Sedan gör du en ändring:

Julia Åhlén
GitHub, Lexicon

```
const name = "Anna";  
  
console.log(`Hello ${name}!`);  
  
console.log("Welcome to my website!");
```

Sedan gör du ytterligare ändringar.

Plötsligt fungerar inte programmet längre.

Hur löser vi problemet?

Ett alternativ är att skapa kopior:

script.js

script-final.js

script-final-2.js

script-final-really-final.js

script-final-really-final-v2.js

Det blir snabbt svårt att hålla ordning på.

Här kommer versionshantering in.

Vad är versionshantering?

Versionshantering innebär att vi sparar information om hur ett projekt förändras över tid.

Vi kan till exempel ha: Version 1 → Version 2 → Version 3 → Version 4

Vi kan se:

- vad som ändrades
- vem som gjorde ändringen
- när ändringen gjordes
- varför ändringen gjordes

Och om något går fel kan vi gå tillbaka och undersöka tidigare versioner.

Git

Git är ett versionshanteringssystem.

Git finns på din dator och håller reda på förändringar i ditt projekt.

Ett förenklat exempel:

Julia Åhlén
GitHub, Lexicon

Projekt

```
|  
├── index.html  
├── style.css  
└── script.js
```

↓

Git

↓

Versionshistorik

Git kan komma ihåg olika versioner av projektet.

GitHub

GitHub är en tjänst där Git-repositories kan lagras online. Git och GitHub är alltså inte samma sak.

Tänk:

Git = versionshantering

GitHub = online-plattform för Git-projekt

GitHub gör det bland annat möjligt att:

- lagra projekt online
- samarbeta med andra
- granska kod
- använda branches
- skapa Pull Requests
- se projektets historik

Branches i Git och GitHub

Vad är en branch?

En **branch** är ett separat utvecklingsspår i ett Git-projekt.

Julia Åhlén
GitHub, Lexicon

När vi arbetar med ett projekt vill vi ofta ha en stabil version som fungerar. Den versionen ligger vanligtvis i en branch som heter: main

Om vi vill utveckla en ny funktion skapar vi istället en ny branch. På så sätt kan vi arbeta med den nya funktionen utan att direkt ändra main.

Ett enkelt exempel

Tänk att vi bygger en JavaScript-to-do-app.

Projektet ser ut så här:

```
main
|
├── index.html
├── script.js
└── style.css
```

Nu vill vi lägga till en funktion som gör det möjligt att radera uppgifter.

Istället för att direkt ändra main skapar vi en ny branch:

```
main
└── feature/delete-task
```

Vi arbetar sedan på:

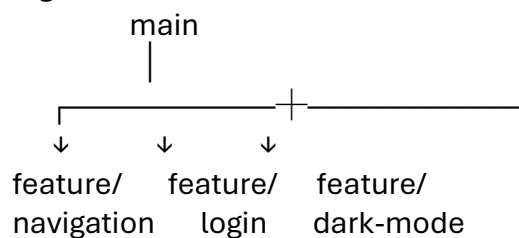
feature/delete-task

När funktionen är färdig kan vi lägga in den i main.

Varför använder man branches?

Branches är framför allt användbara när flera personer arbetar tillsammans. Tänk att fyra studenter arbetar med samma JavaScript-projekt. Student A arbetar med navigation: feature/navigation. Student B arbetar med login: feature/login. Student C arbetar med dark mode: feature/dark-mode. Student D fixar en bugg: bugfix/button.

Alla kan arbeta samtidigt utan att direkt förändra main. Det kan se ut så här:



Julia Åhlén
GitHub, Lexicon

Skapa en branch

Om vi vill skapa en branch för en ny funktion kan vi skriva:

```
git switch -c feature/delete-task
```

Detta gör två saker:

1. Skapar en ny branch.
2. Byter till den nya branchen.

Vi arbetar nu på:

feature/delete-task

och inte direkt på main.

Arbeta på branchen

Nu kan vi ändra vår JavaScript-kod.

Exempel:

```
function deleteTask(task) {  
  // kod för att ta bort uppgiften  
}
```

När vi är nöjda sparar vi förändringen i Git:

```
git add .
```

Sedan skapar vi en commit:

```
git commit -m "Add delete task functionality"
```

Och skickar branchen till GitHub:

```
git push -u origin feature/delete-task
```

Pull Request

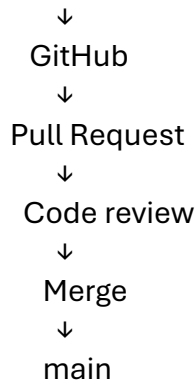
När funktionen är färdig kan vi skapa en **Pull Request** på GitHub. En Pull Request betyder ungefär: "Jag har gjort en förändring. Kan någon granska den och sedan lägga in den i main?" Flödet blir:

feature/delete-task

↓

Push

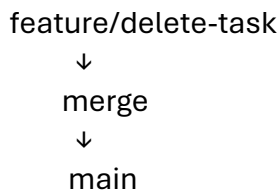
Julia Åhlén
GitHub, Lexicon



Vad betyder merge?

Merge betyder att vi slår ihop förändringar från en branch med en annan branch.

Exempel:



Efter en lyckad merge finns funktionen nu även i main.

Varför inte bara arbeta direkt på main?

Om alla arbetar direkt på main kan det snabbt bli problem.

Exempel:

Student A → ändrar script.js
Student B → ändrar script.js
Student C → ändrar index.html
Student D → råkar ta bort fungerande kod

Det kan bli svårt att veta:

- vem som ändrade vad
- vilka förändringar som hör ihop
- vilka ändringar som är färdiga
- vilka ändringar som ska publiceras

Med branches blir arbetet mer organiserat.

En bra regel

För nybörjare är en bra tumregel: **En uppgift = en branch.**

Julia Åhlén
GitHub, Lexicon

Exempel:

feature/add-login
feature/dark-mode
feature/delete-task
feature/add-score
bugfix/button-color

Branchens namn bör beskriva vad du arbetar med.

Det vanligaste arbetsflödet

När du får en ny programmeringsuppgift:

1. Hämta senaste versionen

↓

2. Skapa en branch

↓

3. Skriv JavaScript-kod

↓

4. Testa koden

↓

5. Commit

↓

6. Push

↓

7. Skapa Pull Request

↓

8. Code review

↓

9. Merge till main

Exempel från ett studentprojekt

Ni bygger en quiz-app i JavaScript. En student får uppgiften: Lägg till en timer på 30 sekunder. Studenten skapar:

git switch -c feature/timer

Studenten programmerar timern och testar den.

Sedan:

`git add .`

`git commit -m "Add quiz timer"`

`git push -u origin feature/timer`

På GitHub skapas sedan en Pull Request: feature/timer → main

En annan student granskar koden. Om allt ser bra ut: Approve → Merge

Julia Åhlén
GitHub, Lexicon

Timern är nu en del av projektets main-branch.

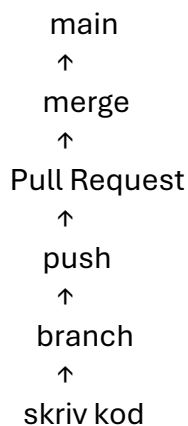
Viktiga begrepp

Begrepp	Betydelse
Branch	Ett separat utvecklingsspår
main	Huvudbranchen, normalt den stabila versionen
Commit	En sparad förändring i Git
Push	Skickar förändringar till GitHub
Pull Request	Förslag att slå ihop en branch med en annan
Code review	Någon granskar koden
Merge	Slår ihop förändringar

Kom ihåg

Det viktigaste är inte att memorera alla Git-kommandon.

Förstå istället denna modell:



Kort sagt: Branch är en egen arbetsyta för en specifik förändring.

Du arbetar på din branch, testar din kod och skapar commits. När du är klar kan du skapa en Pull Request så att förändringen kan granskas innan den mergas till main.

Del 2 Git vs GitHub vs VS Code

30 minuter

Det är viktigt att förstå vilken roll de olika verktygen har.

VS Code

VS Code är din utvecklingsmiljö.

Julia Åhlén
GitHub, Lexicon

Här skriver och testar du din JavaScript-kod.

Exempel:

VS Code → script.js → JavaScript

Git

Git håller reda på förändringar i projektet.

Exempel:

Du ändrar script.js → Git upptäcker ändringen → Du skapar en commit →

Git sparar information om ändringen

GitHub

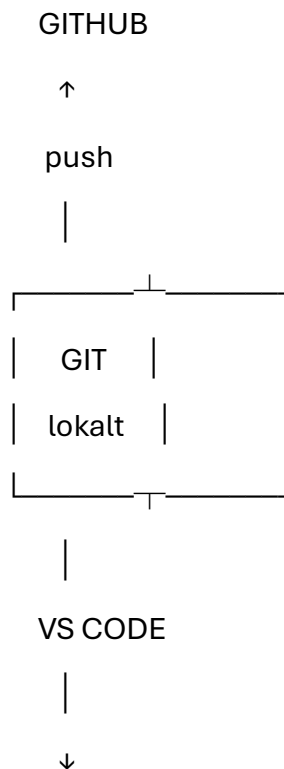
GitHub kan lagra Git-projektet online och hjälpa flera personer att samarbeta.

Exempel:

Din dator → push → GitHub → pull → Din dator

Den viktigaste modellen

Lär dig denna modell:



Julia Åhlén
GitHub, Lexicon

DIN KOD

Vi skriver alltså kod i VS Code. Git håller reda på förändringarna. GitHub kan lagra projektet online.

Del 3 Demo

45 minuter

Nu ska vi skapa ett riktigt JavaScript-projekt.

Steg 1 Skapa ett repository på GitHub

Logga in på GitHub.

Skapa ett nytt repository genom att klicka på

Create repository

Använd exempelvis namnet:

lexicon_project

Välj:

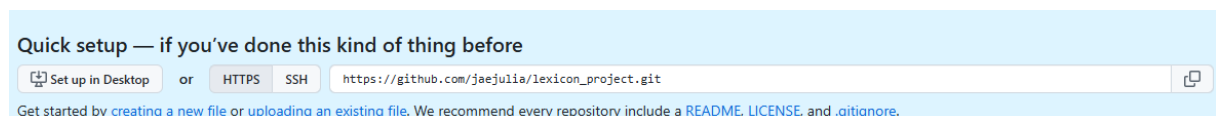
Public eller Private

Det spelar ingen större roll för denna övning.

Skapa repositoryt.

Steg 2 Klona repositoryt

Kopiera repositoryts URL i GitHub



Öppna VS Code.

Skapa en mapp för dina git projekt, tex *my_git*

Öppna terminalen:

Terminal → *New Terminal*

I Terminalen hamnar du i den mappen som du står i för tillfället. Om du vill skapa en annan undermapp, kan du använda kommando `mkdir`, t ex

`mkdir projekt1`

och sedan

Julia Åhlén
GitHub, Lexicon

`cd project1`

När du är nöjd i mappvalet, skriv:

`git clone DIN-REPOSITORY-URL`

Exempel:

`git clone https://github.com/username/lexicon_project.git`

Om du får ett fel som ser ut ungefär så här: *git : The term 'git' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the...*

betyder det att du inte har git installerat i ditt system. Det måste åtgärdas och då skall du gå till [Git - Install for Windows](#) och installera det med standardinställningarna.

Starta om VS Code och öppna Terminal igen.

Skriv `git --version`

Om allt är OK så får du något liknande:

```
(base) PS C:\Users\jae\Downloads\Lektion_lexicontisdag\övningar\Webb10> git --version
git version 2.55.0.windows.4
(base) PS C:\Users\jae\Downloads\Lektion_lexicontisdag\övningar\Webb10> 
```

Gå sedan in i projektmappen, alltså den du skapat tidigare, alternativt öppna den ifrån menyn i VS Code, precis som i tidigare kursen. I Terminalen kan du använda `cd` kommandot för att flytta dig fram i mapparna. Alternativt `cd..` för att gå tillbaka.

`cd lexicon_project`

När du står i projektmappen, klona ditt GitHub projekt med (tänk på att byta till rätt länk):

`git clone https://github.com/username/lexicon_project.git`

Öppna projektet i VS Code genom att skriva:

`code .`

Det kommando kan öppna en extra editor. Det kan du stänga om du inte behöver den. Om du startar VS code igen, skall du inte behöva klona om projektet. Det finns normalt kvar i projektmappen.

Steg 3 Skapa JavaScript-projektet

Skapa filen:

index.html

Julia Åhlén
GitHub, Lexicon

Skriv:

```
<!DOCTYPE html>
<html lang="sv">
<head>
  <meta charset="UTF-8">
  <title>Mitt Git-projekt</title>
</head>
<body>

  <h1>Mitt första Git-projekt</h1>

  <script src="script.js"></script>
</body>
</html>
```

Skapa sedan:

script.js

Skriv:

```
const name = "Anna";

console.log(` Hello ${name}!` );
```

Öppna index.html i webbläsaren.

Öppna webbläsarens Developer Tools och kontrollera Console.

Du ska se:

Hello Anna!

Steg 4 Kontrollera Git

I terminalen:

```
git status
```

Git visar vilka filer som har förändrats.

Du bör se något liknande:

Untracked files:

index.html

script.js

Steg 5 Git add

Julia Åhlén
GitHub, Lexicon

Nu säger vi till Git vilka förändringar vi vill inkludera i nästa commit.

git add.

Kontrollera igen:

git status

Nu bör filerna visas som staged. De är inte *committed* än.

Steg 6 Skapa en commit

En **commit** är en sparad punkt i projektets historik.

Skriv:

git commit -m "Create first JavaScript git project"

Commit-meddelandet ska beskriva vad du gjorde.

Steg 7 Push till GitHub

Nu finns committen lokalt på din dator.

Vi vill skicka den till GitHub:

git push

Du kan behöva autorisera VS Code. I meddelanderutan, klicka på "Sign in with your browser".

Din webbläsare öppnas.

Klicka på den gröna knappen. Authorize ...

När det är klart kan du gå tillbaka till VS Code.

Försök sedan igen med git push.

Gå tillbaka till GitHub och uppdatera sidan.

Nu ska dina filer finnas på GitHub.

Förstå hela flödet

Du har nu gjort:

1. Skrivit kod

↓

2. git status

↓

3. git add

↓

4. git commit

↓

5. git push

Julia Åhlén
GitHub, Lexicon



6. GitHub

VS Code samma sak utan terminalen

VS Code har en egen Git-panel.

Klicka på:

Source Control

Du kan där se:

- vilka filer som ändrats
- vilka filer som är staged
- skapa commits
- pusha
- hämta ändringar

Det är helt okej att använda VS Codes gränssnitt.

Men det är viktigt att förstå vad som händer bakom knapparna.

Exempel:

VS Code-knapp Commit är samma som

"Commit" → git commit

och:

VS Code-knapp "Sync / Push" → git push

Arbetsflödet är följande:

Under Source Control markera filen som har genomgått en ändring

Klicka på + knappen bredvid. Detta är samma som git add .

Skriv ett meddelande i message, t ex vad som hänt med filen och klicka Commit. Ni ka få ett prompt om att skapa **user name** och **user email** för git. Detta görs enklast via Terminal:

git config --global user.name "Julia" Obs! Byt till ditt namn

git config --global user.email din-email@example.com

Testa att köra *Commit* nu och då ändras knappen till **Sync Changes**

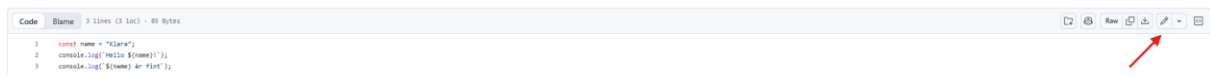
Klicka på den och pusha filen till GitHub. Detta är samma som git push i terminalen.

Julia Åhlén
GitHub, Lexicon

Steg 8 Pull från GitHub

Gå till ditt repository på GitHub. Öppna den senaste versionen av script.js.

Klicka på Edit (pennan) och lägg till:



`console.log("Ändring gjord på GitHub!");`

Klicka på Commit changes knappen, den gröna i högra övre fältet. Nu vet GitHub att det finns en ny version av koden, men inte VS Code.

Gå tillbaka till VS Code och skriv i Terminalen:

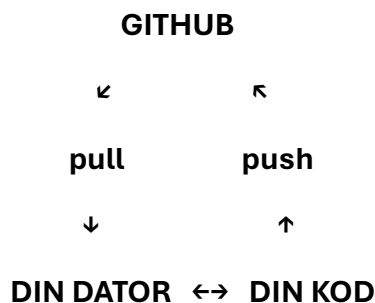
`git status`

`git pull`

Du kommer att se en ny version i Source Control Graph när processen är klar.

Obs! Det kan hända att när du öppnar script.js från Graph blir det Read only, d v s du kan inte direkt ändra i den öppnade filen. Gå till Explorer istället och klicka på script.js därifrån.

Sammanfattning



Ett vanligt grupparbete: Tänk att Student **A** har gjort en ändring och pushat. Student **B** har fortfarande den gamla versionen på sin dator. Student **B** gör *git pull* och får Student **A**'s ändringar. Innan du börjar arbeta med ett gemensamt projekt är det bra att göra: *git pull*.

Del 4 Egen övning

90 minuter

Nu ska du själv skapa och versionshantera ett litet JavaScript-projekt.

Uppgift: "Mitt JavaScript-projekt"

Skapa ett projekt i VS Code, t ex:

my-javascript-project/

Julia Åhlén
GitHub, Lexicon

index.html
script.js

Projektet ska visa information om dig själv, t ex:

Hej! Jag heter Anna.

Jag studerar JavaScript.

Mitt favoritintresse är musik.

Steg 1 Skapa projektet

Skapa ett nytt repository på GitHub.

Döp det till:

my-javascript-project

Klona repositoryt till datorn.

Öppna det i VS Code.

Steg 2 Skapa HTML

Skapa:

index.html

Använd exempelvis:

```
<!DOCTYPE html>
```

```
<html lang="sv">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Om mig</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Om mig</h1>
```

```
  <p id="message"></p>
```

```
  <script src="script.js"></script>
```

```
</body>
```

```
</html>
```

Steg 3 – Skapa JavaScript

Skapa:

script.js

```
const name = "Anna";
```

```
const program = "JavaScript";
```

```
const hobby = "musik";
```


Julia Åhlén
GitHub, Lexicon

```
console.log(` Hej! Jag heter ${name}. ` );  
console.log(` Jag studerar ${program}. ` );  
console.log(` Mitt intresse är ${hobby}. ` );  
Testa programmet.
```

Steg 4 Första committen

Kontrollera status:

```
git status
```

Lägg till filerna:

```
git add .
```

Skapa commit:

```
git commit -m "Create personal JavaScript project"
```

Pusha:

```
git push
```

Kontrollera på GitHub att filerna finns.

Steg 5 Gör en förändring

Ändra JavaScript-programmet.

Lägg exempelvis till en funktion:

```
function introduce(name, program, hobby) {  
    return ` Hej! Jag heter ${name}, studerar ${program} och gillar ${hobby}. ` ;  
}  
console.log(introduce(name, program, hobby));
```

Testa att programmet fungerar.

Steg 6 Skapa ytterligare en commit

Kontrollera:

```
git status
```

Sedan:

```
git add .
```

```
git commit -m "Add introduction function"
```

Och:

```
git push
```

Steg 7 Gör en tredje förändring

Lägg till ytterligare information.

```
const city = "Gävle";
```

Julia Åhlén
GitHub, Lexicon

Ändra sedan funktionen så att staden också visas.

Skapa en ny commit:

`git add .`

`git commit -m "Add city information"`

`git push`

Kontrollpunkt

När du är klar ska ditt projekt ha minst **tre commits**.

På GitHub ska du kunna se ungefär:

Add city information

Add introduction function

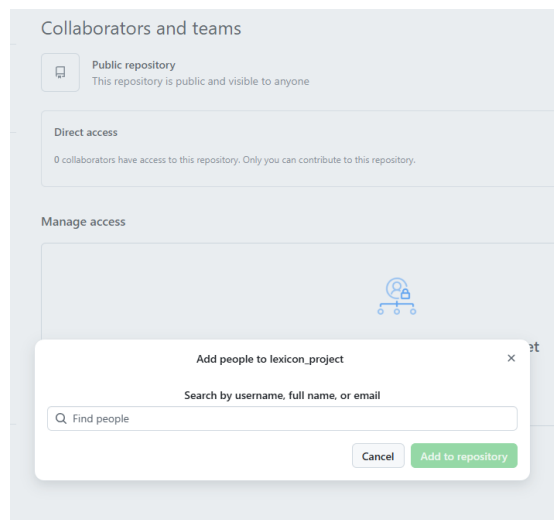
Create personal JavaScript project

Du har nu skapat en enkel versionshistorik.

Steg 8 Grupparbete. Gå till varsin grupp och följ instruktionerna.

Välj en ansvarig medlem i gruppen. Denne skall skapa ett nytt repository på GitHub, t ex *group-javascript-project*.

Den ansvarige skall sedan gå till Settings för repositoryt och lägga *till Collaborators-Add people*. Fyll i användarnamn för gruppmedlemmarna och de skall i sin tur acceptera inbjudan vanligtvis under Notiser-Inbjudan på GitHub, alt via email länkar. Alltså, alla skall kopplas till ett GitHub projekt. Lägg till även lärare jaejulia. Jag skall kolla upp era projekt i slutet av dagen.



Som nästa steg skall alla öppna VS Code och skapa en ny mapp för projektet.

Leta efter länken under repository i GitHub och kopiera den. Den länken skall kunna delas ut till alla av projektansvarige som skapat GitHub repositoryt.

Öppna Terminal och kлона GitHub projekt till VS Code (använd gruppens länk):

`git clone https://github.com/student1/group-javascript-project.git`

Julia Åhlén
GitHub, Lexicon

Skapa tre filer i VS Code: index.html, script.js och style.css. Fyll i dem med nedanstående koder:

index.html

```
<!DOCTYPE html>
<html lang="sv">
<head>
  <meta charset="UTF-8">
  <title>Gruppens projekt</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Vårt JavaScript-projekt</h1>
  <p id="message">Välkommen!</p>
  <script src="script.js"></script>
</body>
</html>
```

style.css

```
body {
  font-family: Arial, sans-serif;
  text-align: center;
}
```

script.js

```
console.log("JavaScript fungerar!");
```

Därefter commitar Projektansvarige till GitHub genom att mata in dessa rader:

```
git add .
git commit -m "Create basic project"
git push
```

Nu skall varje gruppmedlem skapa en egen branch att arbeta på. Bestäm vem skall få vad:

Person 1, projektansvarige:

Skapa en egen branch med

```
git switch -c feature/project
```

när detta är klart, kommer du inte att se feature/project i Explorer, utan kan se det längst ner i VS Code. Du arbetar alltså i din gren nu, inte på hela main.

förbättra exempelvis

```
<h1>Vårt fantastiska JavaScript-projekt</h1>
```

Utför push till GitHub:

```
git add .
```

Julia Åhlén
GitHub, Lexicon

git commit -m "Improve project introduction"
git push -u origin feature/project

Person 2:

Skapa en egen branch

git switch -c feature/css

Kolla att du står i feature/css genom att skriva git status. Du bör se feature/css. Gå till css och förbättra design.

```
body {  
    font-family: Arial, sans-serif;  
    text-align: center;  
    background-color: #f0f4f8;  
    color: #333;  
}  
h1 {  
    color: #2563eb;  
}  
button {  
    padding: 10px 20px;  
    font-size: 16px;  
}
```

Utför push till GitHub:

git add .
git commit -m "Improve page styling"
git push -u origin feature/css

Person 3:

Skapa en egen branch

git switch -c feature/html

Kolla att du står i din branch, skriv *git branch* Du bör se feature/html.

Lägg till en knapp i HTML

`<button id="myButton">Klicka här</button>`

Utför push till GitHub:

git add .
git commit -m "Add button to page"
git push -u origin feature/html

Person 4:

Skapa en egen branch

Julia Åhlén
GitHub, Lexicon

`git switch -c feature/javascript`

Se till att du står i din branch, skriv `git branch` Det bör visa `feature/javascript`
Gå till `script.js` och skapa funktion som reagerar på knapptryckning.

```
const button = document.querySelector("#myButton");
button.addEventListener("click", () => {
  console.log("Button clicked!");
});
```

Utför push till GitHub:

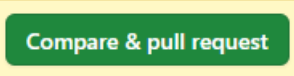
`git add .`

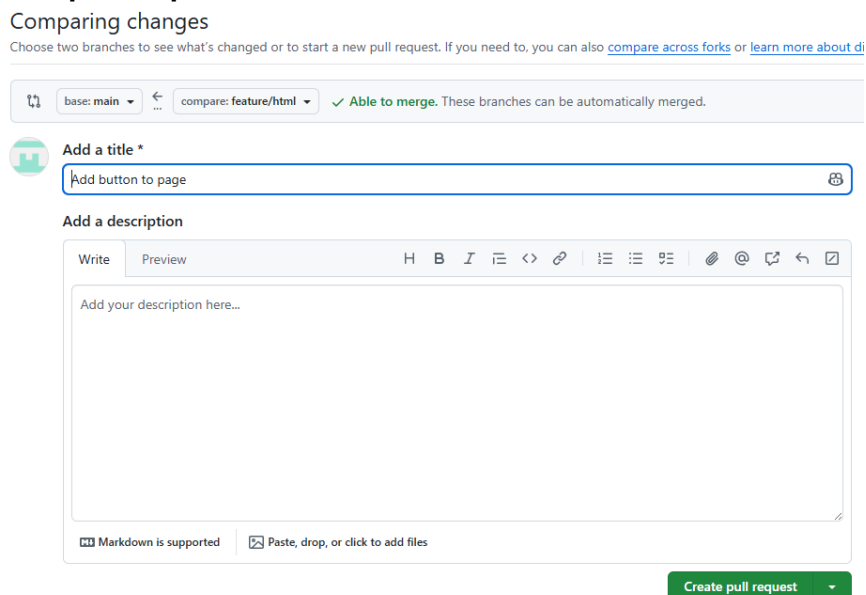
`git commit -m "Add button to page"`

`git push -u origin feature/javascript`

Notera att första gången som ni pushar er branch skall `git push -u origin feature/..` användas. Efter det går det bra med `git push`.

När ni har pushat ändringarna till GitHub skall ni kunna se dem under branches, **All**. Nu skall man kunna lägga till dessa ändringar till de ursprungliga koderna. Nu skall var och en göra nedanstående Pull Request och sedan pull till VS Code. Alltså i tur och ordning.

I GitHub finns nu en knapp  Klicka på den och se till att det ser ut som i nedanstående bild. Du skall alltså överföra ändringarna till main från din branch. Klicka på **Create pull request**.



Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff](#).

base: main ← compare: feature/html ✓ Able to merge. These branches can be automatically merged.

Add a title *

Add button to page

Add a description

Write Preview H B I `< >          `

Add your description here...

Markdown is supported  Paste, drop, or click to add files

Create pull request

Ändringarna är inte inlagda än, de är enbart förfrågade. Nu skall ni kunna öppna varandras pull request och kolla upp koden under Files Changed fliken. Om allt är OK,

Julia Åhlén
GitHub, Lexicon

alla är nöjda klickar man på Approve knappen och först då kan vi utföra **merge** operationen. Klicka på *Merge pull request* och sedan *Confirm*.

Alla skall sedan skriva i Terminalen i VS Code:

git switch main

git pull

Vanliga problem

"git är inte ett kommando"

Git är troligen inte installerat eller finns inte i systemets PATH.

Kontrollera:

git --version

"Jag kan inte pusha"

Kontrollera:

git status

och:

git remote -v

Kontrollera också att du är inloggad/auktoriserad mot GitHub.

"Jag har ändrat filen men GitHub visar inte ändringen"

Kom ihåg:

Ändra fil

↓

git add

↓

git commit

↓

git push

↓

GitHub uppdateras

Att bara ändra filen i VS Code skickar inte automatiskt ändringen till GitHub.

Repetition

Kan du förklara dessa begrepp?

Git

Vad är Git?

Git är ett versionshanteringssystem som håller reda på förändringar i ett projekt.

GitHub

Vad är GitHub?

Julia Åhlén
GitHub, Lexicon

GitHub är en plattform där Git-repositories kan lagras och där utvecklare kan samarbeta.

Repository

Vad är ett repository?

Ett repository är ett projekt som Git håller reda på, inklusive projektets versionshistorik.

Commit

Vad är en commit?

En commit är en sparad förändring i Git-projektets historik.

Push

Vad gör git push?

Skickar lokala commits till GitHub.

Pull

Vad gör git pull?

Hämtar förändringar från GitHub till det lokala projektet.

De viktigaste kommandona

Kommando Vad gör det?

<code>git clone</code>	Hämtar ett repository från GitHub
<code>git status</code>	Visar projektets status
<code>git add</code>	Lägger förändringar i staging
<code>git commit</code>	Sparar en version i Git
<code>git push</code>	Skickar commits till GitHub
<code>git pull</code>	Hämtar förändringar från GitHub

Kom ihåg!

Det viktigaste flödet från dagens lektion är:

ÄNDRA KOD

↓

git status

↓

git add

↓

git commit

↓

git push

↓

GITHUB